

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: DIANA CAROLINA ACUÑA ACUÑA

PARALELO: A

SEMESTRE: Quinto Semestre

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

PERIODO ABRIL 2026 - AGOSTO 2026



Actividades a desarrollar en la Unidad 1

Actividad # Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

Análisis del problema

Autores Principales

Los actores principales identificados en el sistema de Help Desk para la gestión de fallos en un Data Center son el Usuario Final, el Técnico de Soporte y el Administrador de Sistemas. El Usuario Final es la persona que detecta y reporta los incidentes que afectan el funcionamiento del Data Center (hardware, red o software). Por su parte, el Técnico de Soporte se encarga de recibir los tickets asignados, realizar el diagnóstico, aplicar las soluciones correspondientes y actualizar el estado de los incidentes. Finalmente, el Administrador de Sistemas tiene un rol de supervisión y gestión, ya que es responsable de registrar y gestionar los usuarios y técnicos, asignar o reasignar tickets, definir categorías y generar reportes de gestión.

Según Romero y Callupe (2024), en los sistemas Help Desk resulta fundamental una correcta definición de actores, ya que cada uno cumple un rol específico dentro del flujo de atención de incidencias, lo que permite una gestión más ordenada, eficiente y trazable de los tickets. Esta separación de responsabilidades favorece la escalabilidad y la mantenibilidad del sistema, aspectos clave en entornos críticos como un Data Center.

ID	Actor	Descripción	Responsabilidades Principales
A-01	Usuario Final	Empleado o personal que utiliza los servicios del Data Center y detecta fallos.	- Reportar incidentes - Adjuntar evidencias - Consultar estado de sus tickets - Recibir notificaciones
A-02	Técnico de Soporte	Personal técnico especializado en áreas de Hardware, Red o Software.	- Recibir tickets asignados - Diagnosticar y resolver incidentes - Actualizar estado y agregar notas - Marcar tickets como resueltos
A-03	Administrador de Sistemas	Responsable de la supervisión y gestión general del sistema Help Desk.	- Gestionar usuarios y técnicos - Asignar / reasignar tickets - Definir categorías y prioridades - Generar reportes y métricas - Realizar auditorías

Requerimientos Funcionales

ID	Requerimiento Funcional	Descripción	Actor Principal	Prioridad
RF-01	Registro de Incidentes	Permitir la creación de tickets con título, descripción detallada y evidencia (capturas)	Usuario Final	Alta



RF-02	Categorización de Fallos	Clasificar automáticamente o manualmente el incidente en: Hardware, Red, Software u Otros	Usuario Final / Admin	Alta
RF-03	Asignación de Técnico	Asignar técnico especializado según categoría, disponibilidad y carga de trabajo	Administrador / Sistema	Alta
RF-04	Gestión de Estados del Ticket	Controlar el ciclo de vida: Abierto → Asignado → En Progreso → Resuelto → Cerrado	Técnico / Administrador	Alta
RF-05	Actualización de Tickets	Permitir agregar notas, diagnósticos, soluciones y evidencias durante la resolución	Técnico	Alta
RF-06	Consulta y Seguimiento	Búsqueda y filtrado de tickets por estado, categoría, técnico, fecha, etc.	Todos los actores	Alta
RF-07	Notificaciones	Enviar notificaciones automáticas por cambio de estado o asignación (email y/o dashboard)	Sistema	Media
RF-08	Gestión de Técnicos	Registrar técnicos con sus especialidades y disponibilidad	Administrador	Media
RF-09	Reportes y Métricas	Generar reportes de tiempo de resolución, tickets por categoría, SLA, etc.	Administrador	Media
RF-10	Historial y Auditoría	Registrar todas las acciones realizadas en cada ticket (trazabilidad)	Administrador	Alta

Requerimientos No Funcionales

ID	Requerimiento No Funcional	Descripción	Prioridad
RNF-01	Escalabilidad	El sistema debe soportar al menos 500 tickets simultáneos y crecimiento futuro	Alta
RNF-02	Mantenibilidad	Código modular, bien documentado y fácil de extender (nuevas categorías o integraciones)	Alta
RNF-03	Trazabilidad de Versiones	Uso obligatorio del modelo Gitflow en repositorio GitHub	Alta
RNF-04	Seguridad	Autenticación, autorización por roles (RBAC) y protección de datos sensibles	Alta
RNF-05	Disponibilidad	Sistema disponible 24/7 con recuperación ante fallos	Alta



RNF-06	Usabilidad	Interfaz intuitiva y responsive	Alta
RNF-07	Rendimiento	Tiempo de respuesta < 2 segundos en operaciones principales	Media
RNF-08	Portabilidad	Independencia de plataforma (ejecutable en Windows, Linux, Cloud)	Media
RNF-09	Auditoría y Cumplimiento	Registro completo de acciones para auditorías internas	Alta
RNF-10	Integración	Posibilidad de integración futura con sistemas de monitoreo del Data Center	Media

Diseño del sistema

Repositorio

DESARROLLO_DE_SISTEMAS_INFORMATICOS Public

main 1 Branch 0 Tags

Go to file

Code

Initial commit

ea51a83 · 26 minutes ago 1 Commit

README.md Initial commit 26 minutes ago

README

DESARROLLO_DE_SISTEMAS_INFORMATICOS

Sistema de Gestión de Incidentes (Help Desk) para infraestructura de servidores. Implementa patrones de diseño de software (Singleton, Observer, Factory Method) y sigue el modelo Gitflow para control de versiones. Incluye análisis de arquitectura, diagramas UML y documentación de diseño orientado a objetos con principios SOLID.

About

Sistema de Gestión de Incidentes (Help Desk) para infraestructura de servidores. Implementa patrones de diseño de software (Singleton, Observer, Factory Method) y sigue el modelo Gitflow para control de versiones. Incluye análisis de arquitectura, diagramas UML y documentación de diseño orientado a objetos con principios SOLID.

Readme

Activity

0 stars

0 watching

0 forks

Creación del repositorio en GitHub



Gitflow

Branches · Diana716 · x · Universidad Técnica · x · Sección: Mater... · x · Git Credential Man... · x · Sistema Help Desk... · x · +

← → ↻ github.com/Diana716269/DESARROLLO_DE_SISTEMAS_INFORMATICOS/branches Centro educativo

Diana716269 / DESARROLLO_DE_SISTEMAS_INFORMATICOS

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Branches

New branch

Overview Yours Active Stale All

Search branches...

Default

Branch	Updated	Check status	Behind / Ahead	Pull request
main	2 hours ago		Default	

Your branches

Branch	Updated	Check status	Behind / Ahead	Pull request
feature/patrones	10 minutes ago		0 / 0	
feature/diseño-sistema	20 minutes ago		0 / 3	
feature/analisis-problema	1 hour ago		0 / 3	
develop	2 hours ago		0 / 0	

creación de ramas (master, develop y ramas feature/ para avances).

Diagrama de casos de uso

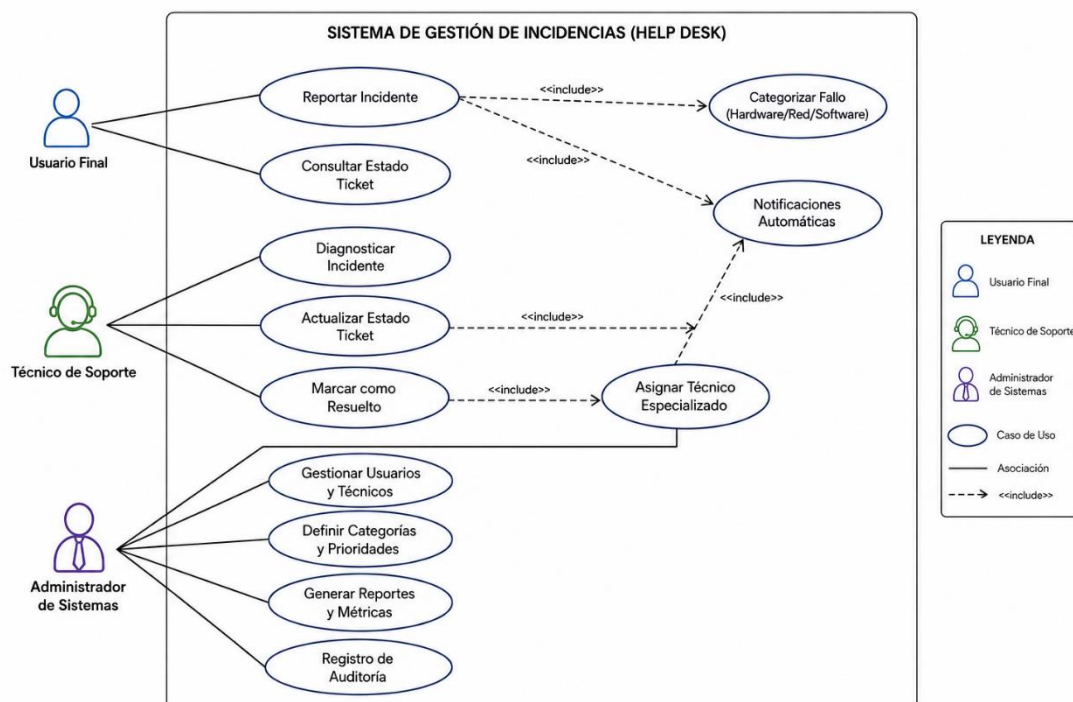
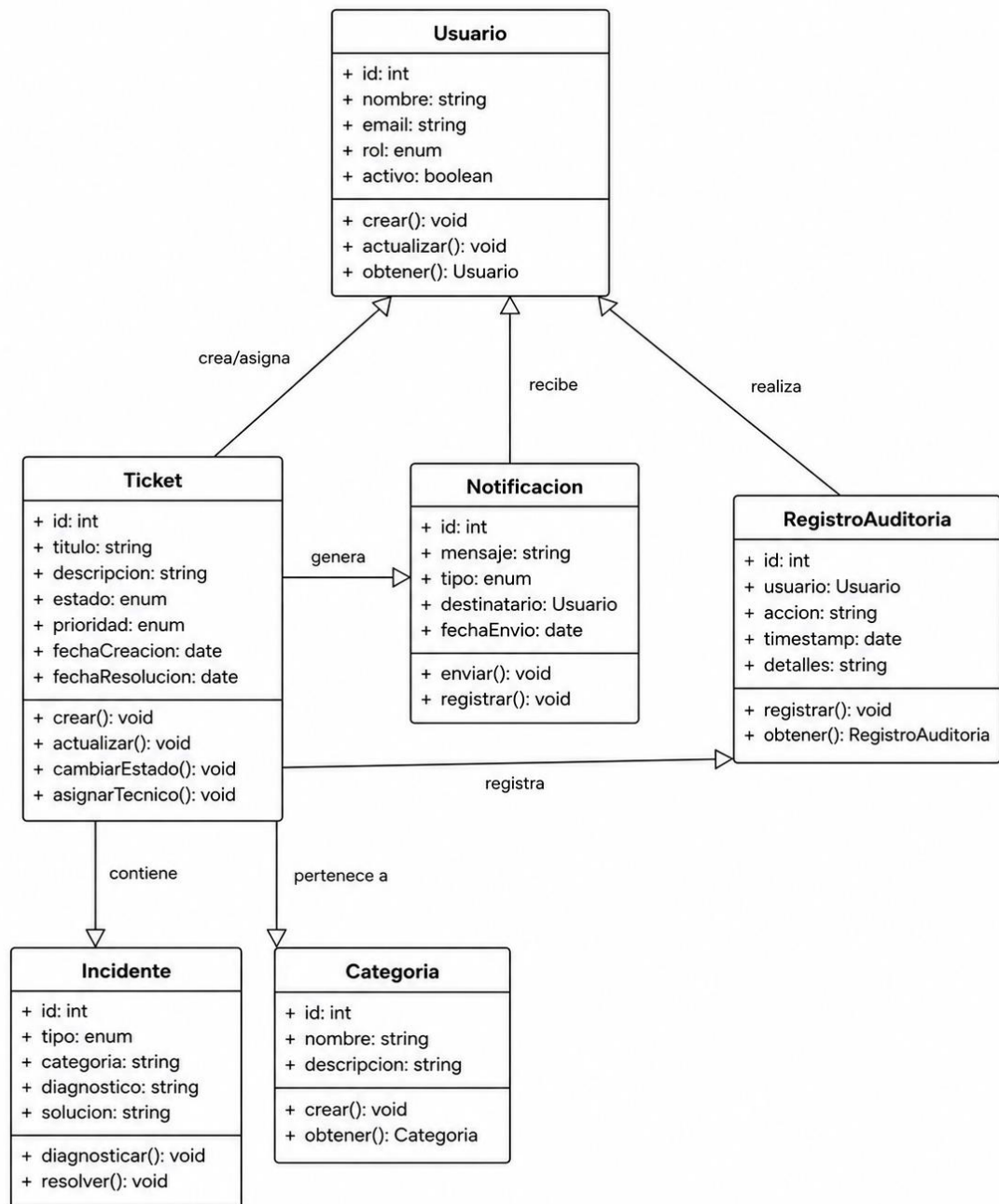
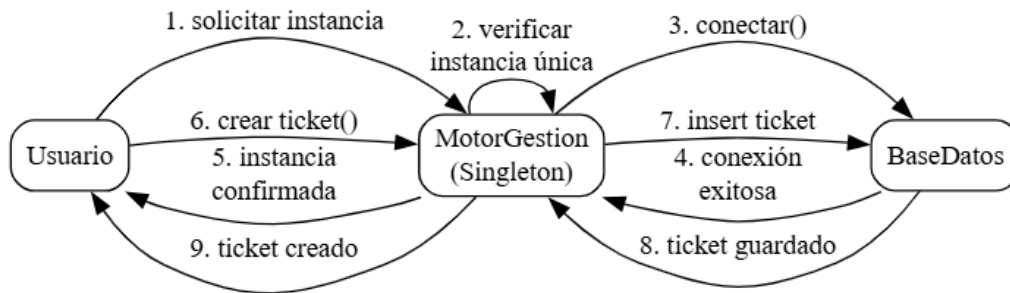




Diagrama de clases UML



Arquitectura general del sistema



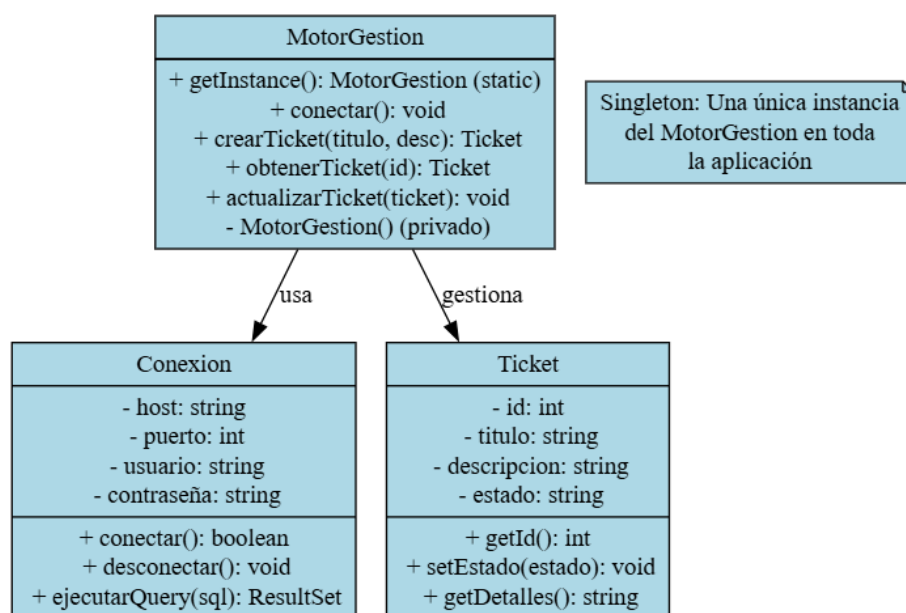
Aplicación de patrones de diseño

Para lograr una arquitectura robusta, mantenible y escalable en el sistema de Help Desk orientado a la gestión de fallos en un Data Center, se han implementado los siguientes cuatro patrones de diseño:

1. Singleton

Propósito: Garantizar que solo exista una única instancia de la clase TicketManager y de la conexión a la base de datos durante toda la ejecución de la aplicación.

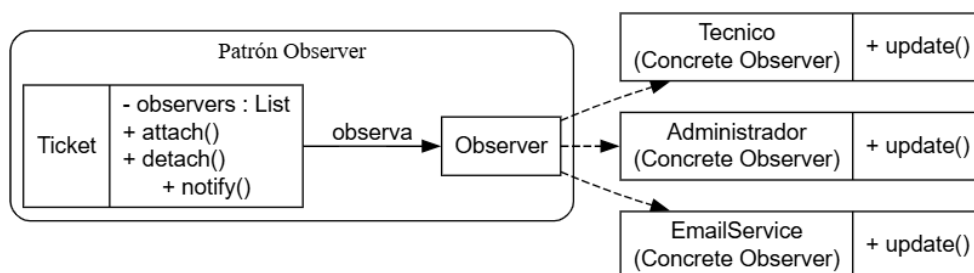
Problema que resuelve: En un entorno de Data Center donde múltiples usuarios y técnicos pueden crear o actualizar tickets simultáneamente, es crítico evitar múltiples conexiones a la base de datos o instancias conflictivas del motor de gestión de tickets. El patrón Singleton asegura el control centralizado de recursos compartidos, previene el consumo innecesario de memoria y mantiene la consistencia de los datos.



2. Observer (Publicador-Suscriptor)

Propósito: Notificar automáticamente a los técnicos y usuarios cuando ocurre un evento relevante (creación de ticket, cambio de estado o asignación).

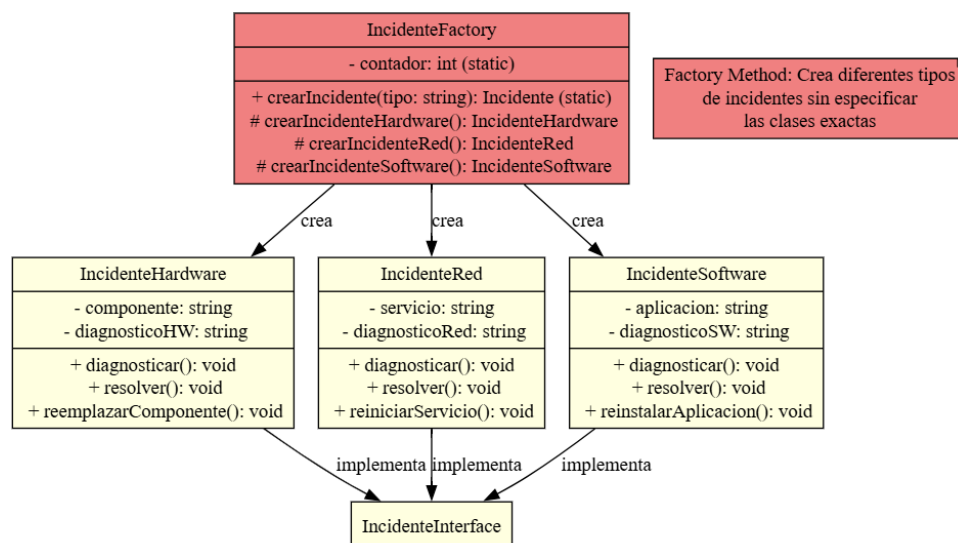
Problema que resuelve: El sistema requiere que los técnicos sean informados en tiempo real de nuevos incidentes asignados a su especialidad sin necesidad de consultar constantemente el sistema. El patrón Observer desacopla el objeto sujeto (Ticket) de los observadores (técnicos, administrador y sistema de correo), permitiendo una arquitectura reactiva y extensible (fácil agregar nuevos canales de notificación como Microsoft Teams o SMS).



3. Factory Method

Propósito: Crear diferentes tipos de incidentes según su categoría (HardwareIncident, NetworkIncident, SoftwareIncident) sin especificar la clase concreta en el código cliente.

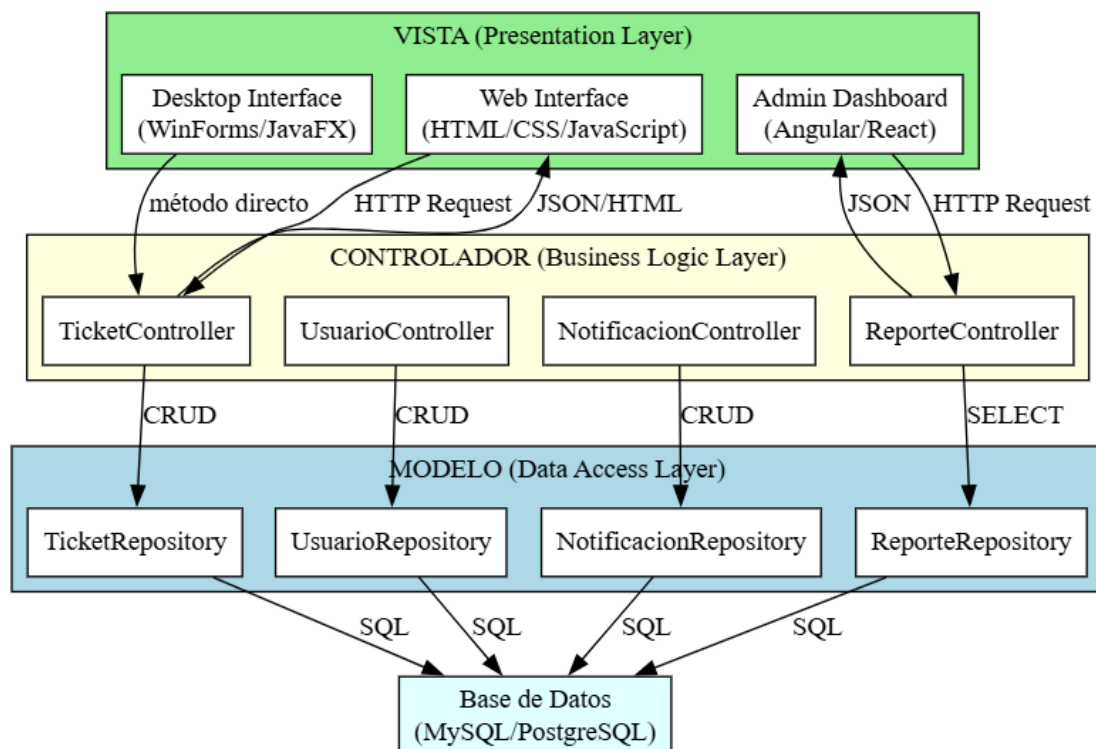
Problema que resuelve: Cada tipo de fallo posee atributos, validaciones y flujos de resolución distintos. El Factory Method centraliza la lógica de creación, mejora la mantenibilidad y cumple con el principio Open/Closed, permitiendo agregar nuevas categorías de incidentes (ej. Seguridad o Aplicaciones) sin modificar el código existente.



4. MVC (Model-View-Controller)

Propósito: Separar la lógica de negocio, la presentación y el control de flujos en capas independientes.

Problema que resuelve: Proporciona una estructura clara y organizada del sistema, facilitando el trabajo en equipo, las pruebas unitarias y futuras modificaciones en la interfaz de usuario sin afectar la lógica de negocio. Es especialmente útil en aplicaciones web donde se requiere una clara separación de responsabilidades.

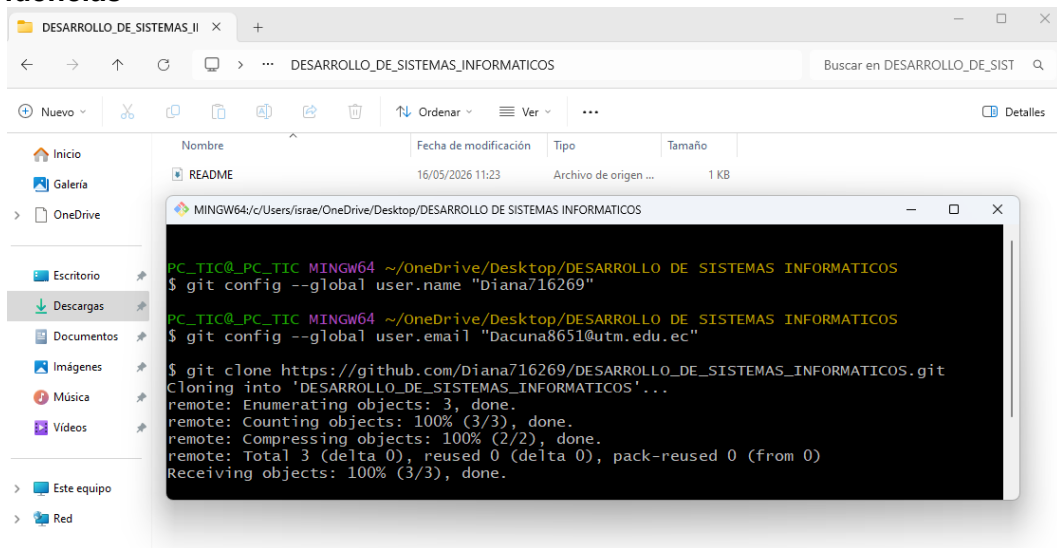


Justificación del Uso de Gitflow

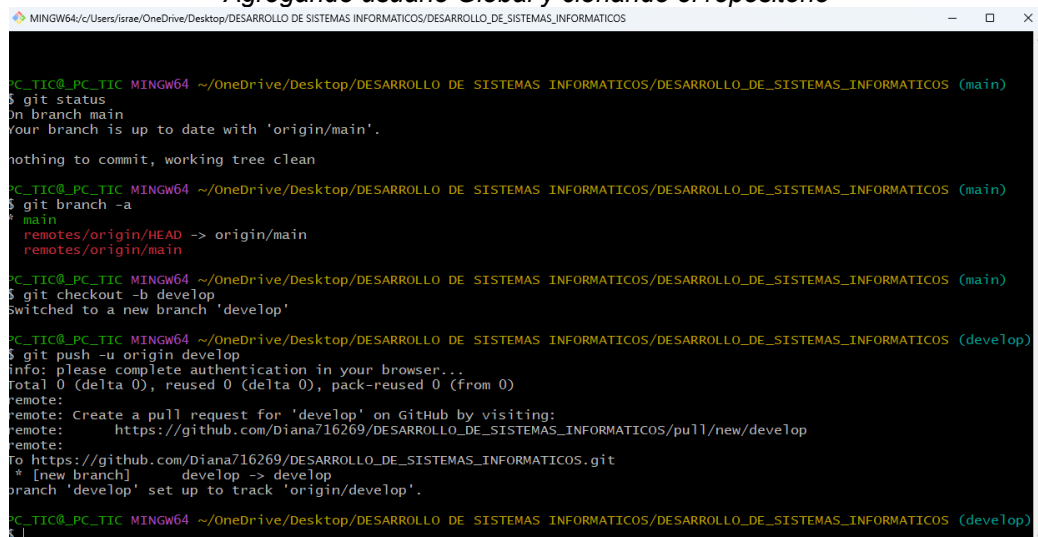
El desarrollo del sistema se ha gestionado mediante el modelo Gitflow en un repositorio de GitHub. Este flujo de trabajo fue seleccionado porque permite un control ordenado de las versiones, separación clara entre desarrollo, pruebas y producción, y facilita el trabajo colaborativo.



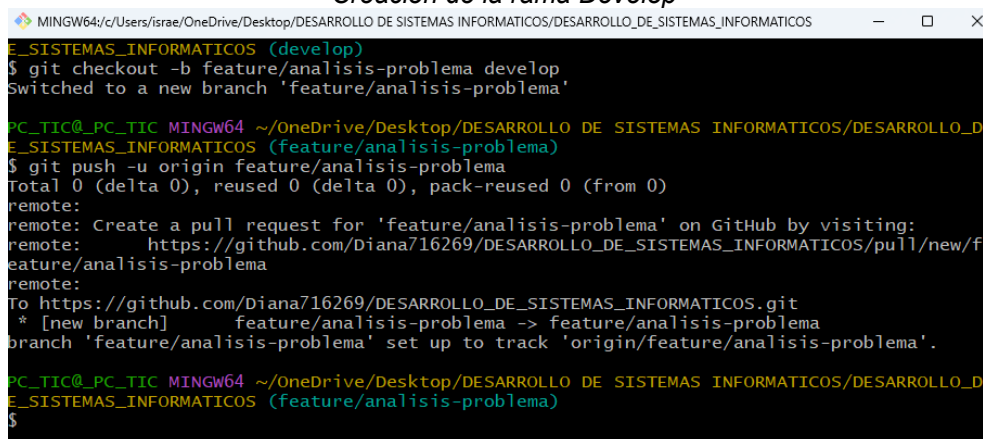
Evidencias



Agregando usuario Global y clonando el repositorio



Creación de la rama Develop



Creación de la rama feature



Commits

History for `DESARROLLO_DE_SISTEMAS_INFORMATICOS` / `Actividad_1` on `develop`

All users

All time

Commits on May 16, 2026

Merge pull request #2 from Diana716269/feature/diseño-sistema

Verified

773cef0

11 minutes ago

Diana716269 authored 11 minutes ago

Merge pull request #3 from Diana716269/feature/patrones

Verified

107d79d

11 minutes ago

Diana716269 authored 11 minutes ago

docs: agregar documentacion completa patrones Singleton, Observer y Factory Method

9458a72

37 minutes ago

Diana716269 committed 37 minutes ago

docs: agregar arquitectura general del sistema integrando flujo de versiones Gitflow

9470c22

1 hour ago

Diana716269 committed 1 hour ago

docs: agregar diagrama de clases UML con principios SOLID

d0aab4f

1 hour ago

Diana716269 committed 1 hour ago

docs: agregar diagrama de casos de uso - interacciones personal IT con infraestructura

d0e596f

1 hour ago

Diana716269 committed 1 hour ago

docs: agregar requerimientos no funcionales (escalabilidad, mantenibilidad, trazabilidad)

e8bc214

2 hours ago

Diana716269 committed 2 hours ago

docs: agregar requerimientos funcionales (crear ticket, categorizar fallo, asignar tecnico)

788ae76

2 hours ago

Diana716269 committed 2 hours ago

docs: agregar identificacion de actores principales (Usuario final, Tecnico, Admin)

2acd001

2 hours ago

Diana716269 committed 2 hours ago

End of commit history for this file

Historial de commits

DESARROLLO_DE_SISTEMAS_INFORMATICOS / Actividad_1

Files

- Actividad_1
 - analisis
 - Analisis_Incidentes.docx
 - diseño
 - diagramas
 - Patrones_Diseño.docx
 - README.md

DESARROLLO_DE_SISTEMAS_INFORMATICOS / Actividad_1

Diana716269 Merge pull request #2 from Diana716269/feature/diseño-si... 773cef0 · 12 minutes ago

This branch is 10 commits ahead of main.

Name	Last commit message	Last commit date
..		
analisis	docs: agregar requerimientos no funcionales (esc...	2 hours ago
diseño	Merge pull request #2 from Diana716269/featur...	12 minutes ago

Actividad 1 en la rama develop

Enlace del Proyecto en GitHub

https://github.com/Diana716269/DESARROLLO_DE_SISTEMAS_INFORMATICOS.git

Bibliografía

Romero, S. P., & -Callupe, F. J. M. (2024). Sistema Help Desk: una solución para la gestión de incidencias. Gaceta Científica, 10(2), 76-81. <https://doi.org/10.46794/gacien.10.2.2188>